Module 6: Lesson 2

Application of autoencoders: Extracting features from stock returns

Outline

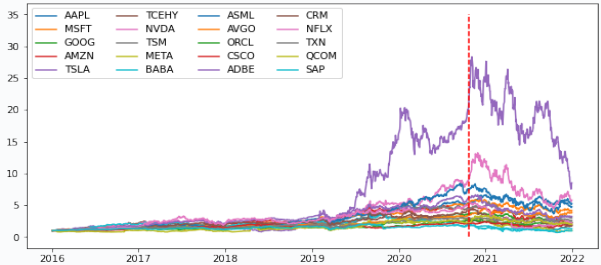
- Data
- PCA decomposition
- Building and training an autoencoder

Data

In this lesson, we illustrate the use of PCA and autoencoders for financial data.

Our data under analysis are the daily stock returns from a sample of 20 tech stocks from Jan 2017 to Dec 2022.

- The training sample covers the period up to Oct 20th, 2021 (80% of the full sample).

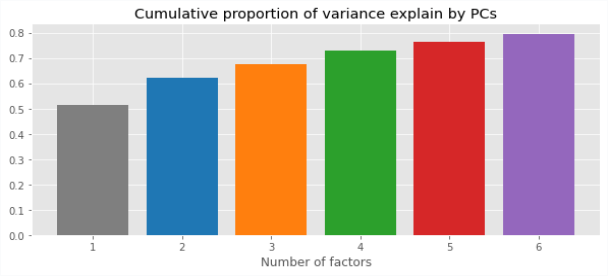


PCA decomposition

The PCA decomposition of the returns data over the training sample shows that roughly 50% of the variation arises from the first principal component.

- In other words, 50% of the variation in returns comes from a single factor that generates comovement across stocks (i.e., "the market").

Considering 5 additional factors leads to explaining an additional 30% of all the variation in returns.



Building an autoencoder in TensorFlow

We build, via "subclassing," a linear autoencoder, which in TensorFlow is designed as follows:

- A Dense layer (encoder) that compresses the input into a latent vector with dimension "bottleneck_dim."
- A Dense layer (decoder) that reconstructs the original input from the latent space.

```
class autoencoder(tf.keras.Model):
  def __init__(self, output_dim, bottleneck_dim = 1):
    super(autoencoder, self).__init__()
    self.output_dim = output_dim
    self.bottleneck_dim = bottleneck_dim
    initializer = tf.keras.initializers.GlorotUniform()
    # Build the encoder layer
    self.encoder = tf.keras.Sequential([
      tf.keras.layers.Dense(bottleneck_dim,
                            kernel_initializer=initializer,
                            bias_initializer = tf.keras.initializers.Zeros()
                           )
    ])
    # Build the decoder layer
    self.decoder = tf.keras.Sequential([
      tf.keras.layers.Dense(output_dim,
                            kernel_initializer=initializer,
                            bias_initializer = tf.keras.initializers.Zeros()
                           )
    ])
  def call(self, x):
    encoded = self.encoder(x)
    decoded = self.decoder(encoded)
    return decoded
```

Training the autoencoder

We train the linear autoencoder with a "bottleneck" dimension equal to 3.

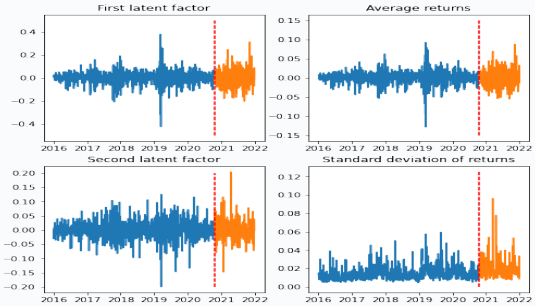
We set apart the last 20% of the training sample for validation purposes, using an early-stopping criterion.

- We use the default batch size of 32, train for 1,000 epochs, and set the patience equal to 50 epochs. We use the Adam optimizer with a learning rate of 10^{-5}.

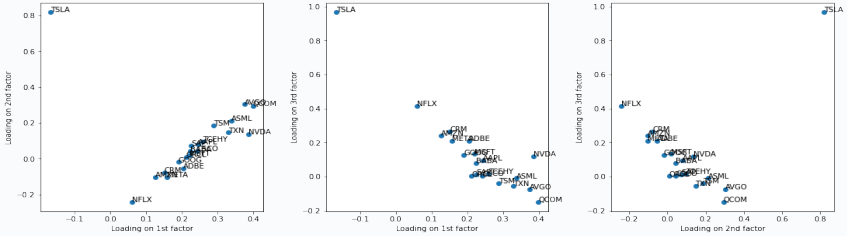
Disclaimer: The results below refer to a particular initialization of the trainable parameters and may vary if other seeds are used.

The reconstruction error in training reaches 32%, close to the lower bound set by PCA.

The first latent factor explains the average returns. The second latent factor seems to explain the dispersion of returns.



Trained loadings



A large portion of the cross-sectional dispersion in stock returns, captured by the second and third factors, is actually driven by the behavior of Tesla's stock price.

Stocks whose returns correlate more with the average return are also more exposed to the cross-sectional dispersion factor.

The third factor seems to capture a feature of stocks that are less exposed to market variations and cross-sectional dispersion.

Summary of Lesson 2

In Lesson 2, we have:

- Used PCA and autoencoders to extract the main latent factors that drive stock returns.
- Built and trained autoencoder models in TensorFlow via "subclassing."

⇒ **References for this lesson:**

Chapter 7 in Ranjan, C. *Understanding Deep Learning*.

TO DO NEXT: Please go to the Jupyter Notebook associated with Lesson 2.

In the next lesson, we will describe a variety of autoencoder architectures and several refinement tools.
See you there!